

Arquitectura de Ejecución en ROS 2: Del Comando de Terminal al Módulo Python

Jesús Leonardo Cárdenas Martínez
Unidad Tamaulipas
Cinvestav
Ciudad Victoria, México
jesus.cardenas@cinvestav.mx

José Gabriel Ramírez Torres
Unidad Tamaulipas
Cinvestav
Ciudad Victoria, México
grtorres@cinvestav.mx

Abstract—Este documento describe de manera detallada y progresiva la arquitectura de ejecución de ROS 2 (Robot Operating System 2), siguiendo el flujo completo de información desde el comando escrito en la terminal hasta la instanciación de un módulo Python dentro de un nodo. Se explican los mecanismos internos de cada capa: el entorno de ejecución del sistema operativo, el rol del ejecutable `ros2` como script Python, el sistema de launch files y sus mecanismos de sustitución diferida, el ciclo de vida de arranque de un nodo, el Parameter Server, y el loop de eventos en tiempo real basado en DDS. El objetivo es que un lector con conocimientos de Python y Linux pero sin experiencia previa en ROS 2 pueda comprender no solo *qué* ocurre, sino *por qué* ocurre, y sea capaz de trazar el recorrido de cualquier argumento desde la terminal hasta su uso en el código.

Index Terms—ROS 2, launch files, nodos, variables de entorno, Parameter Server, DDS, arquitectura de software robótico

I. INTRODUCCIÓN

ROS 2 (Robot Operating System 2) es un middleware de código abierto para el desarrollo de sistemas robóticos [1]. A diferencia de un sistema operativo convencional, ROS 2 no gestiona hardware directamente. Provee una capa de abstracción que permite a procesos independientes, llamados nodos, comunicarse entre sí mediante tópicos, servicios y acciones, sin conocer los detalles de implementación del otro.

Una característica que suele generar confusión en desarrolladores que se inician en ROS 2 es que el sistema no oculta sus capas internas. Para usarlo correctamente es necesario entender cómo el sistema operativo gestiona procesos, qué son las variables de entorno, y cómo un argumento escrito en la terminal termina disponible como un valor numérico dentro de una clase Python.

Este documento traza ese recorrido de forma exhaustiva. Las primeras dos secciones cubren el entorno del sistema operativo: variables de entorno, el comando `source` y la naturaleza del ejecutable `ros2`. La sección sobre el launch file explica los mecanismos de sustitución diferida y la creación de descriptores de nodo. La sección sobre el arranque del nodo describe el ciclo de inicialización, el Parameter Server y las comunicaciones DDS. Las secciones

finales cubren el paso de parámetros a clases Python puras y el loop de eventos en tiempo real.

A lo largo del documento se utiliza como hilo conductor el siguiente comando, representativo de los experimentos de control de formación desarrollados en este trabajo:

Listing 1. Comando de ejemplo utilizado a lo largo del documento.

```
1 ros2 launch simulador demo_pva.launch.py
  \
2   n_rays_used:=5 vmax:=0.8
```

Este comando lanza una simulación en Gazebo con nodos de control que implementan el Polígono de Velocidades Admisibles (PVA) [2]. Cada argumento pasado en la terminal recorrerá las cuatro capas descritas antes de llegar al constructor de la clase PVA.

II. EL ENTORNO DE EJECUCIÓN

Antes de que el sistema ROS 2 pueda ser invocado, el sistema operativo debe saber dónde encontrarlo. Esta sección explica la infraestructura sobre la que todo lo demás descansa.

A. Distribuciones de ROS 2

ROS 2 sigue un esquema de versiones llamado distribuciones, análogo al de Ubuntu o Debian. Cada distribución tiene un nombre en clave formado por un adjetivo y un animal, ambos iniciando con la misma letra, en orden alfabético. Las principales distribuciones son:

TABLE I
DISTRIBUCIONES DE ROS 2 Y SU SOPORTE.

Distribución	Año	SopORTE
Foxy Fitzroy	2020	Expirado
Galactic Geochelone	2021	Expirado
Humble Hawksbill	2022	LTS hasta 2027
Iron Irwini	2023	Expirado
Jazzy Jalisco	2024	LTS hasta 2029

Las distribuciones LTS (Long Term Support) tienen soporte garantizado por cinco años y son las recomendadas para proyectos de investigación y producción. **Humble Hawksbill** es la distribución LTS activa más extendida

al momento de este trabajo y la que se utiliza en todos los experimentos descritos.

Cada distribución se instala en su propio directorio dentro de `/opt/ros/`. Si se tuvieran Humble y Jazzy instaladas simultáneamente, coexistirían en `/opt/ros/humble/` y `/opt/ros/jazzy/` sin interferir entre sí. La distribución activa en una terminal es aquella cuyo `setup.bash` fue sourceado, y la variable `ROS_DISTRO` indica cuál es:

Listing 2. Verificación de la distribución activa.

```
1 echo $ROS_DISTRO
2 # humble
```

B. Dónde vive ROS 2

ROS 2 Humble se instala en `/opt/ros/humble/`. El directorio `/opt/` es el lugar estándar en Linux para software opcional instalado en el sistema, es decir, software que no viene con el OS pero que se agrega encima. La estructura interna es análoga a la del propio sistema operativo:

Listing 3. Estructura de instalación de ROS 2 Humble.

```
1 /opt/ros/humble/
2 |-- bin/          <- ejecutables: ros2,
   rviz2, gazebo...
3 |-- lib/          <- librerías compiladas y
   paquetes Python
4 |-- include/      <- headers de C++
5 |-- share/        <- recursos y launch files
   del sistema
6 '-- setup.bash <- el script que se
   sourcea
```

El workspace del usuario replica exactamente esta estructura, pero contiene únicamente los paquetes desarrollados por él:

Listing 4. Estructura del workspace tras compilar con `colcon build`.

```
1 ws3/install/
2 |-- control_nodes/
3 |   |-- bin/
4 |   '-- lib/python3.10/site-packages/
5 |-- simulador/
6 '-- setup.bash <- encadenador: sourcea
   ROS2 + workspace
```

C. Variables de entorno

Una variable de entorno es un par `NOMBRE=valor` almacenado en la memoria del proceso actual. No pertenece a ningún programa en particular, sino al entorno del proceso: la tabla de configuración global que el sistema operativo mantiene para cada proceso en ejecución.

`PATH` es la variable que contiene una lista de directorios separados por `:` donde el OS busca ejecutables. Si `/opt/ros/humble/bin` no está en `PATH`, el comando `ros2` devuelve `command not found`. `PYTHONPATH` cumple la misma función para módulos Python: sin esta variable configurada, la instrucción `import rclpy` falla con `ModuleNotFoundError`. `AMENT_PREFIX_PATH` es utilizada

internamente por ROS 2 para localizar los paquetes instalados mediante `ament_index`. Finalmente, `ROS_DISTRO` contiene el nombre de la distribución activa y es leída por algunos paquetes para ajustar su comportamiento. Todas pueden inspeccionarse con:

Listing 5. Inspección de variables de entorno relevantes.

```
1 echo $PATH          # rutas donde el OS
   busca ejecutables
2 echo $PYTHONPATH    # rutas donde Python
   busca módulos
3 echo $ROS_DISTRO    # distribución de ROS2
   activa
4 env                 # muestra todas las
   variables
```

D. Qué es `source` y por qué es necesario

Cuando se ejecuta un script de shell de la manera convencional, el OS crea un proceso hijo, ejecuta el script en ese proceso hijo, y al terminar el proceso hijo desaparece llevándose consigo cualquier cambio a las variables de entorno. La terminal del usuario no es afectada.

Listing 6. Diferencia entre ejecutar y sourcear un script.

```
1 # Ejecutar normal: modifica un proceso
   hijo que desaparece
2 bash setup.bash      # los cambios a PATH
   se pierden
3
4 # Sourcear: ejecuta las líneas
   directamente en la terminal
5 source setup.bash    # los cambios a PATH
   persisten
```

El comando `source` (equivalente al punto `.` en bash) ejecuta las líneas del script en el proceso actual de la terminal, como si se hubieran escrito directamente. Las modificaciones a `PATH` y `PYTHONPATH` quedan en ese proceso y son heredadas por todos los programas que se lancen desde ahí.

E. Qué hace `setup.bash` internamente

El archivo `ws3/install/setup.bash` generado por `colcon build` actúa como encadenador: primero sourcea ROS 2, luego el workspace del usuario. Su función central es invocar un script Python auxiliar que inspecciona los paquetes instalados en `ws3/install/` y genera dinámicamente los comandos `export` necesarios:

Listing 7. Mecanismo central de `local_setup.bash` (simplificado).

```
1 # El script Python lee ws3/install/ y
   genera los export
2 _cmds="$(python3 local_setup_util_sh.py
   sh_bash)"
3
4 # eval ejecuta esas líneas en la terminal
   actual
5 eval "${_cmds}"
6
7 # El resultado equivale aproximadamente a
   :
```

```

8 export PATH="/ws3/install/control_nodes/
  bin:$PATH"
9 export PYTHONPATH="/ws3/install/
  control_nodes/lib/\
10 python3.10/site-packages:$PYTHONPATH"

```

Esto es importante: `source setup.bash` no instala nada ni configura nada de forma permanente. Solo modifica las variables de entorno de la terminal actual. Al cerrar esa terminal, las modificaciones desaparecen. Por eso es necesario sourcear en cada terminal nueva.

F. El shebang y el PATH

El ejecutable `ros2` es un archivo de texto. Su primera línea contiene:

Listing 8. Primera línea del ejecutable `ros2`.

```

1 #!/usr/bin/env python3

```

Esta línea se llama shebang. Cuando Linux intenta ejecutar un archivo con permisos de ejecución, lee su primera línea: si comienza con `#!`, usa lo que sigue como intérprete y le pasa el archivo como argumento. `/usr/bin/env python3` no apunta a Python directamente; llama a `env`, que busca `python3` en el PATH actual. Como ya se sourceó ROS 2, ese `python3` tiene acceso a todos los módulos de ROS 2 a través de `PYTHONPATH`.

En consecuencia, escribir `ros2 launch ...` en la terminal es exactamente equivalente a:

Listing 9. Equivalencia entre el comando y su expansión real.

```

1 python3 /opt/ros/humble/bin/ros2 launch \
2   simulador demo_pva.launch.py \
3   n_rays_used:=5 vmax:=0.8

```

III. EL EJECUTABLE ROS2

A. Cómo el OS entrega los argumentos

Cuando el sistema operativo ejecuta cualquier programa, le entrega todos los tokens escritos en la terminal como una lista de strings llamada `sys.argv`. Para el comando del Listado 1, esa lista es:

Listing 10. `sys.argv` recibido por el ejecutable `ros2`.

```

1 sys.argv = [
2   'ros2',
3   'launch',
4   'simulador',
5   'demo_pva.launch.py',
6   'n_rays_used:=5',
7   'vmax:=0.8'
8 ]

```

Esta es la frontera más baja del sistema. El OS no sabe si 5 es un entero, un float, o una cadena de texto. Todo es un string. Esta restricción se propaga hacia arriba y obliga a conversiones explícitas de tipo en cada capa.

B. Estructura modular de `ros2`

El ejecutable `ros2` es una CLI (Command Line Interface) modular. Cada subcomando (`launch`, `run`, `topic`, `node`, `param`) es un plugin Python separado que se carga bajo demanda. Al recibir `sys.argv[1] = 'launch'`, el ejecutable carga el plugin `ros2launch` y le delega el resto de la ejecución. Esto permite extender `ros2` con subcomandos personalizados sin modificar el ejecutable principal.

C. Parsing de los argumentos `:=`

El plugin `ros2launch` utiliza `argparse` de Python para separar el nombre del paquete y el archivo `launch`. Todo lo que queda después de esos dos campos es tratado como overrides de argumentos del `launch file`. La convención `clave:=valor` no pertenece a Python ni al OS; es una convención que ROS 2 definió y parsea manualmente mediante string splitting:

Listing 11. Parsing de overrides de argumentos (simplificación del

```

1 launch_arguments = {}
2 for token in remaining_args:
3     if ':= ' in token:
4         key, value = token.split(':= ', 1)
5         launch_arguments[key] = value #
6             siempre string
7 # Resultado para el comando del Listado
8 1:
9 # launch_arguments = {'n_rays_used': '5',
10    'vmax': '0.8'}

```

El `split(':= ', 1)` asegura que si el valor contiene `:=` (por ejemplo, una ruta), solo se divide en la primera ocurrencia. El resultado es un diccionario de strings que se entrega al sistema de `launch` como condiciones iniciales.

D. Localización del `launch file` con `ament_index`

Una vez parseados los argumentos, `ros2launch` necesita localizar el archivo `demo_pva.launch.py` dentro del paquete `simulador`. Para ello usa `ament_index`: una base de datos de paquetes instalados que vive en `AMENT_PREFIX_PATH`. Cada paquete, al compilarse con `colcon build`, registra su nombre y ubicación en esa base de datos. `ament_index` hace el equivalente a:

Listing 12. Localización del `launch file` (simplificado).

```

1 from ament_index_python.packages import \
2   get_package_share_directory
3
4 pkg_dir = get_package_share_directory('
5   simulador')
6 # -> '/ws3/install/simulador/share/
7   simulador'
8
9 launch_file = pkg_dir + '/launch/demo_pva
10    .launch.py'

```

Finalmente, `ros2launch` importa ese archivo como módulo Python normal, mediante

importlib.import_module, y busca la función `generate_launch_description()`.

IV. EL LAUNCH FILE

A. `generate_launch_description()`

Esta función es el punto de entrada que `ros2launch` busca en el archivo importado. Debe retornar un objeto `LaunchDescription`: una lista ordenada de acciones que el sistema de launch procesará en secuencia.

Listing 13. Estructura típica de `generate_launch_description()`.

```
1 def generate_launch_description():
2     return LaunchDescription([
3         DeclareLaunchArgument(
4             'n_rays_used',
5             default_value='0',
6             description='Número de rayos LiDAR activos'
7         ),
8         DeclareLaunchArgument(
9             'vmax',
10            default_value='1.0',
11            description='Velocidad lineal máxima (m/s)'
12        ),
13        OpaqueFunction(function=
14            launch_setup
15    )]
```

B. `DeclareLaunchArgument`

Esta acción registra que el launch file acepta un argumento con ese nombre, haciéndolo visible para herramientas como `ros2 launch -show-args`. Al mismo tiempo establece su valor: si el argumento llegó desde la terminal mediante `:=`, lo usa; si no llegó, aplica el `default_value`. En ambos casos el valor permanece como string. `DeclareLaunchArgument` no evalúa ni convierte el valor bajo ninguna circunstancia. La conversión de tipo es responsabilidad del código que lo consume.

C. El objeto `context`

El `context` es el objeto central del sistema de launch. Es una estructura de datos que ROS 2 mantiene durante toda la ejecución y que contiene los valores resueltos de todos los argumentos declarados, el estado global del launch (qué procesos están activos, qué acciones han sido ejecutadas), y el mecanismo de resolución de Substitutions. No es un objeto que el usuario crea directamente; ROS 2 lo construye y lo pasa a las funciones que lo necesitan.

D. `OpaqueFunction`: lógica Python arbitraria

`OpaqueFunction` le indica al sistema de launch que, cuando llegue a esa posición de la lista, llame a la función Python especificada con el `context` actual. Es el mecanismo que permite ejecutar lógica arbitraria (bucles, cálculos, condicionales) dentro del launch file, algo que las acciones declarativas estándar no permiten. La función recibe el `context` y debe retornar una lista de acciones adicionales.

E. `LaunchConfiguration` y Substitutions

Listing 14. Resolución de argumentos dentro de `launch_setup`.

```
1 def launch_setup(context, *args, **kwargs):
2     n_rays = int(
3         LaunchConfiguration('n_rays_used')
4         .perform(context)
5     )
6     vmax = float(
7         LaunchConfiguration('vmax')
8         .perform(context)
9     )
```

`LaunchConfiguration('n_rays_used')` no lee el valor inmediatamente. Crea un objeto `Substitution`: una promesa diferida que sabe cómo resolver su valor cuando se le proporcione un `context`. Esta distinción es relevante porque en el sistema de launch algunas acciones se construyen antes de que todos los argumentos estén resueltos.

`.perform(context)` resuelve esa promesa: consulta el `context`, recupera el string '5' y lo retorna. La conversión `int(...)` es responsabilidad del usuario, ya que ROS 2 nunca convierte tipos.

F. Creación del descriptor de nodo

Listing 15. Creación del descriptor de nodo con parámetros.

```
1 Node(
2     package='control_nodes',
3     executable='navigate_individual_pva',
4     namespace='robot0',
5     parameters=[{
6         'n_rays_used': n_rays, # int 5
7         'vmax': vmax, # float
8         0.8
9     }],
10 )
```

El objeto `Node` no lanza el proceso todavía. Es un descriptor que el sistema de launch procesará cuando llegue su turno. Al procesarlo, serializa el diccionario `parameters` a YAML y lo pasa al proceso como argumento de línea de comandos. Lo que el sistema operativo termina ejecutando es equivalente a:

Listing 16. Comando real que el OS ejecuta al procesar el descriptor

```
1 navigate_individual_pva \
2 --ros-args \
3 -p n_rays_used:=5 \
4 -p vmax:=0.8 \
5 -r __ns:=/robot0
```

El diccionario Python se convirtió en flags de CLI. Esta es la segunda frontera de serialización del flujo: Python dict → YAML → argumentos de proceso.

V. ARRANQUE DEL NODO

A. El mecanismo `entry_points`

El ejecutable `navigate_individual_pva` no es un archivo Python que el usuario crea manualmente. Es un

script generado automáticamente por `setuptools` durante `colcon build`, basado en la declaración en `setup.py`:

Listing 17. Declaración del ejecutable en `setup.py`.

```
1 entry_points={
2     'console_scripts': [
3         'navigate_individual_pva_=_',
4         'control_nodes.
5             navigate_individual_pva:main'
6     ],
7 }
```

Esta declaración indica que el ejecutable `navigate_individual_pva` debe llamar a la función `main()` del módulo `control_nodes.navigate_individual_pva`. `colcon build` genera un script wrapper en `ws3/install/control_nodes/bin/` que hace exactamente eso. Por ello, al agregar un nuevo ejecutable a un paquete es necesario recompilar: el wrapper no existe hasta que `colcon` lo genera.

B. La función `main()`

Listing 18. Función `main()` típica de un nodo ROS 2.

```
1 def main(args=None):
2     rclpy.init(args=args)
3     node = NavigateIndividual()
4     rclpy.spin(node)
5     node.destroy_node()
6     rclpy.shutdown()
```

`rclpy.init(args=args)` inicializa la comunicación con el middleware DDS (Data Distribution Service): la capa de transporte subyacente que gestiona el descubrimiento de nodos, la transmisión de mensajes entre tópicos y el Parameter Server. Sin esta llamada, ninguna funcionalidad de ROS 2 está disponible. Recibe `args=None` en la mayoría de los casos porque los argumentos `-ros-args` ya fueron procesados por el runtime de ROS 2 antes de llegar a `main`.

`rclpy.spin(node)` bloquea la ejecución del proceso en un loop de eventos (detallado en la Sección VII). El proceso permanece en esta línea hasta que recibe una señal de terminación (Ctrl+C o SIGINT).

C. El constructor `__init__` del nodo

Listing 19. Constructor de un nodo ROS 2.

```
1 class NavigateIndividual(Node):
2     def __init__(self):
3         super().__init__('
4             navigate_individual')
5         # ...
```

La llamada `super().__init__('navigate_individual')` registra este nodo en el grafo ROS 2 con el nombre indicado. A partir de este momento el nodo es visible para el resto del sistema: `ros2 node list` lo mostrará. El nombre del nodo es independiente del nombre del ejecutable. Son dos identificadores distintos.

D. Declaración y lectura de parámetros

Listing 20. Declaración y lectura de parámetros en el nodo.

```
1 # Declarar: registrar el parametro con su
2   valor por defecto
3 self.declare_parameter('n_rays_used', 0)
4 self.declare_parameter('vmax', 1.0)
5
6 # Leer: recuperar el valor almacenado
7 n_rays = self.get_parameter('n_rays_used')
8         .value # int 5
9 vmax    = self.get_parameter('vmax').value
10         # float 0.8
```

`declare_parameter` cumple dos funciones simultáneamente. Por un lado, registra en el Parameter Server interno del nodo que existe un parámetro con ese nombre y ese tipo por defecto. Por otro, si el valor ya llegó a través de `-ros-args -p n_rays_used:=5` (generado por el sistema de launch), lo almacena con ese valor en lugar del default.

`get_parameter` accede al Parameter Server y recupera el valor. `.value` lo devuelve en el tipo Python apropiado (int, float, str, bool, list). El tipo fue inferido automáticamente al parsear el YAML generado por el launch file. Esta es la última frontera de serialización: YAML → Python typed value.

E. Las tres fronteras de serialización

El valor 5 escrito en la terminal recorre tres conversiones antes de estar disponible en Python:

TABLE II
FRONTERAS DE SERIALIZACIÓN DEL ARGUMENTO `n_rays_used:=5`.

Frontera	De	A	Mecanismo
Terminal → launch	string '5'	string '5'	<code>split(':=')</code>
Launch → proceso	Python int 5	YAML 5	<code>ros-args -p</code>
Proceso → nodo	YAML 5	Python int 5	<code>.value</code>

En la primera frontera no hay conversión. Todo permanece como string. La conversión a int la realiza el usuario en el launch file con `int(LaunchConfiguration(...).perform(context))`. En la tercera frontera, el YAML se deserializa automáticamente y el tipo es inferido.

VI. DEL NODO AL MÓDULO

A. Instanciación de módulos Python puros

Una vez recuperados los valores del Parameter Server, el nodo los usa para construir los objetos que implementan la lógica de control. En este punto el código abandona por completo el dominio de ROS 2:

Listing 21. Instanciación del módulo PVA desde el nodo.

```
1 from control_nodes.algorithms.pva import
2   PVA
3
4 self.pva = PVA(
5     n_rays_used=n_rays, # int 5
6     v_max=vmax, # float 0.8
```



```

6     d_safe=self.get_parameter('d_safe').
      value,
7     d_influence=self.get_parameter('
      d_influence').value,
8     xi=self.get_parameter('xi').value,
9     rp=0.25,
10 )

```

`n_rays` es un entero Python normal. La clase PVA no sabe nada de ROS 2, no sabe que existe un launch file, no sabe que hubo una terminal. Recibe un argumento de constructor como cualquier clase Python.

B. La clase PVA como Python puro

Listing 22. Constructor de la clase PVA.

```

1 class PVA:
2     def __init__(self,
3                 d_safe=0.3,
4                 d_influence=1.0,
5                 xi=1.0,
6                 rp=0.25,
7                 v_max=0.5,
8                 w_max=0.5,
9                 n_rays_used=None):
10
11         self.d_safe      = d_safe
12         self.d_influence = d_influence
13         self.xi          = xi
14         self.rp          = rp
15         self.v_max       = v_max
16         self.w_max       = w_max
17         self.n_rays_used = n_rays_used

```

Desde aquí en adelante es orientación a objetos Python estándar. La separación entre el código de ROS 2 (el nodo) y la lógica de la aplicación (el módulo) es una práctica arquitectónica deliberada: permite probar la clase PVA de forma independiente, sin necesitar un contexto ROS 2 activo.

C. Por qué esta separación es importante

El nodo actúa como adaptador: su responsabilidad es recibir datos de la infraestructura ROS 2 (tópicos, parámetros) y entregarlos a los módulos de lógica de control en los tipos correctos. Los módulos implementan los algoritmos sin acoplarse al middleware. Esto tiene consecuencias concretas. En cuanto a testabilidad, los módulos pueden probarse con `unittest` sin inicializar `rclpy`. En cuanto a portabilidad, el mismo módulo puede usarse en otro sistema de middleware cambiando solo el nodo adaptador. En cuanto a claridad, la complejidad de ROS 2 queda confinada al nodo y la lógica de control es matemáticamente pura.

VII. EJECUCIÓN EN TIEMPO REAL

A. El loop de eventos: `rclpy.spin()`

Una vez que el nodo está construido y todos sus suscriptores, publicadores y timers registrados, la llamada `rclpy.spin(node)` entrega el control al loop de eventos de ROS 2:

Listing 23. Comportamiento conceptual de `rclpy.spin()`.

```

1 # Pseudocódigo del loop de eventos
2 while rclpy.ok():
3     esperar_evento()           # bloquea
4     ejecutar_callback()       # procesa
5     el evento
6     # repetir

```

El proceso permanece bloqueado en este loop indefinidamente. Los callbacks y timers son la única manera en que el código del usuario vuelve a ejecutarse después del `__init__`.

B. Suscripciones

Una suscripción registra un callback que ROS 2 llama automáticamente cada vez que llega un mensaje en el tópico indicado:

Listing 24. Registro de una suscripción a odometría.

```

1 self.odom_sub = self.create_subscription(
2     Odometry,           # tipo del
3                         mensaje
4     'odom',             # nombre del
5                         tópico
6     self.odom_callback, # función a
7                         llamar
8     10                  # queue size
9 )

```

El argumento 10 es el tamaño de la cola: si los mensajes llegan más rápido de lo que el callback los procesa, se acumulan hasta 10 mensajes antes de empezar a descartarlos. DDS gestiona la entrega del mensaje desde el publicador (el nodo de odometría de Gazebo) hasta este callback, incluso si están en procesos diferentes.

C. Timers

Un timer registra un callback que se ejecuta periódicamente:

Listing 25. Registro de un timer de control a 20 Hz.

```

1 self.timer = self.create_timer(
2     0.05,           # periodo en
3                     segundos (20 Hz)
4     self.control_loop # función a
5                       llamar
6 )

```

D. Modelo de concurrencia

Por defecto, `rclpy.spin()` utiliza un ejecutor de un solo hilo (`SingleThreadedExecutor`), lo que significa que los callbacks y timers no se ejecutan en paralelo: el loop de eventos los pone en cola y los ejecuta uno a uno. Esto tiene dos consecuencias importantes. La primera es favorable: la lectura y escritura sobre variables compartidas (`self.x`, `self.ranges`) es segura sin necesidad de locks. La segunda es una limitación: un callback lento retrasa todos los demás, por lo que el loop de control debe ser eficiente. Si se necesita paralelismo real, existe `MultiThreadedExecutor`, pero requiere manejo explícito de concurrencia.

DDS (Data Distribution Service) es la capa de transporte subyacente de ROS 2. Es un estándar industrial de comunicación publicador-suscriptor. Cuando un nodo arranca, DDS anuncia su existencia en la red y otros nodos que suscriben al mismo tópico se conectan automáticamente, sin configuración manual de IPs o puertos. Para la transmisión, DDS serializa los mensajes al formato wire, los transmite (localmente vía memoria compartida, o en red vía UDP) y los deserializa en el receptor. Además, gestiona políticas de calidad de servicio (confiabilidad, durabilidad, historial) configurables por tópico.

El usuario de ROS 2 no interactúa con DDS directamente; `rclpy` lo abstrae completamente. Sin embargo, entender que DDS existe explica por qué los nodos se descubren automáticamente y por qué es posible comunicar nodos en máquinas diferentes con solo configurar `ROS_DOMAIN_ID`.

VIII. RESUMEN DEL FLUJO COMPLETO

La Tabla III condensa el recorrido completo del argumento `n_rays_used:=5` a través de las cuatro capas del sistema.

TABLE III
FLUJO COMPLETO DEL ARGUMENTO `n_rays_used:=5` DESDE LA TERMINAL HASTA EL MÓDULO.

Capa	Forma del valor	Mecanismo	Responsable
Terminal	string '5'	<code>sys.argv</code> / <code>split(':=')</code>	OS / ros2
Launch file	string '5' → int 5	<code>.perform(ctx) + int(...)</code>	Usuario
Proceso	YAML <code>n_rays: 5</code>	<code>-ros-args -p</code>	ROS 2
Nodo	int 5	<code>get_parameter.value</code>	<code>rclpy</code>
Módulo	int 5	argumento constructor	Python

El diagrama siguiente ilustra las entidades involucradas en cada nivel y los mecanismos de cruce entre ellos:

Listing 26. Diagrama de flujo Terminal → Módulo.

```

1  TERMINAL
2  "n_rays_used:=5 vmax:=0.8"
3  | sys.argv + split(':=')
4  v
5  LAUNCH FILE [
6    generate_launch_description]
7  DeclareLaunchArgument  <- registra y
   almacena
8  OpaqueFunction         <- logica
   Python
9  .perform(context)      <- resuelve
   el string
10 int('5') = 5           <- conversion
   : usuario
11 Node(parameters={...}) <- descriptor
   (no proceso)
   | serializa a YAML, pasa como --
   ros-args -p
12 v
13 PROCESO [navigate_individual_pva --ros
   -args ...]
14 rclpy.init()           <- inicia DDS
15 Node.__init__()        <- registra
   en el grafo
16 declare_parameter()    <- almacena
   en param server
17 get_parameter().value  <- recupera
   como int
   | argumento de constructor
   Python normal
18 v
19 MODULO [class PVA]
20 self.n_rays_used = 5    <- Python
   puro

```

A. Observaciones finales

Tres conclusiones emergen del análisis de este flujo.

Todo empieza como string. La restricción del OS de que `sys.argv` es siempre una lista de strings se propaga hacia arriba. ROS 2 nunca convierte tipos automáticamente. Esa responsabilidad recae en el usuario dentro del launch file.

Cada capa tiene su propio mecanismo de serialización. El cruce entre capas requiere una conversión de formato: string splitting en la terminal, YAML al pasar al proceso, deserialización en el Parameter Server. Comprender estos mecanismos permite depurar errores de tipo que de otro modo son opacos.

El módulo de lógica no sabe que existe ROS 2. La arquitectura de separación nodo-módulo no es accidental: confina la complejidad del middleware a una capa de adaptación y deja el código de control matemáticamente puro, testeable y portable.

REFERENCES

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

- [2] J. G. Ramírez-Torres and F. Arambula-Cosío, "Collision avoidance in the velocity space for non-holonomic vehicles," in *Proceedings of the IEEE International Conference on Robotics and Automation*. IEEE, 2001, pp. 2195–2200.